Ministerul Educaţiei şi Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică şi Microelectronică

Laboratory work 2:
Digital Logic Circuits in LogiSim

Elaborated:
st. gr. FAF-243 Kushnirenko Ecaterina

Verified:
asist. univ. Grosu Olga

Chişinău - 2026

Contents

1 PURPOSE OF THE WORK

The objective of this laboratory is to design, build, and test a variety of digital logic circuits inside the LogiSim simulation environment. The exercises are organized into three difficulty tiers — easy, medium, and hard — and they span a wide range of topics in digital electronics: from elementary logic gates and LED indicators, through flip-flops and multiplexers, all the way up to multi-bit arithmetic units and number-system converters. By constructing each circuit by hand, connecting every wire and placing every component manually, we develop a practical intuition for how abstract Boolean algebra translates into working hardware.

The laboratory also reinforces several theoretical concepts covered in lectures. Building an SR flip-flop out of NOR gates, for instance, makes the idea of feedback-based memory much more tangible than a truth table alone. Similarly, cascading D flip-flops into a delay line or a register shows exactly how sequential logic differs from combinational logic — the role of the clock edge becomes impossible to ignore when you watch signals propagate through the simulation one tick at a time.

A total of 17 exercises were selected so that the accumulated point value reaches exactly 10.0. The breakdown is 2 easy exercises at 0.2 points each, 3 medium exercises at 0.4 points each, and 12 hard exercises at 0.7 points each, giving $2 \times 0.2 + 3 \times 0.4 + 12 \times 0.7 = 0.4 + 1.2 + 8.4 = 10.0$. The selection was also made to touch as many different subcategories as possible — registers, comparators, encoders, decoders, arithmetic units, bitwise operators, and conversion circuits all appear at least once.

2 EXERCISES

This section presents every completed exercise together with a screenshot of the circuit as it appears in LogiSim and a detailed explanation of how the circuit works, what components were used, and why they were connected the way they were. The exercises are grouped by difficulty level.

2.1 Easy exercises

2.1.1 1E — NOT Gate (Basic Logic Gates)

The NOT gate, also called an inverter, is the simplest possible logic gate. It takes a single binary input and produces the opposite value at the output. When the input is at logic high (1), the output drops to logic low (0), and when the input is at logic low the output rises to logic high. In terms of Boolean algebra, if the input is A, then the output is $\overline{A}$.

The truth table is trivial: there are only two possible input values. For $A = 0$ the output is 1, and for $A = 1$ the output is 0. Despite its simplicity, the NOT gate is one of the most important building blocks in digital design. It is used inside virtually every other gate (NAND, NOR, XNOR all include an inversion stage), and it is essential for constructing complementary signals that many sequential circuits depend on.

In LogiSim the circuit consists of three components: an input pin on the left, a NOT gate in the middle, and an output pin on the right. The wiring is a straight horizontal line passing through the inverter. When the simulation is running, clicking the input pin toggles it between 0 and 1, and the output pin immediately reflects the inverted value.

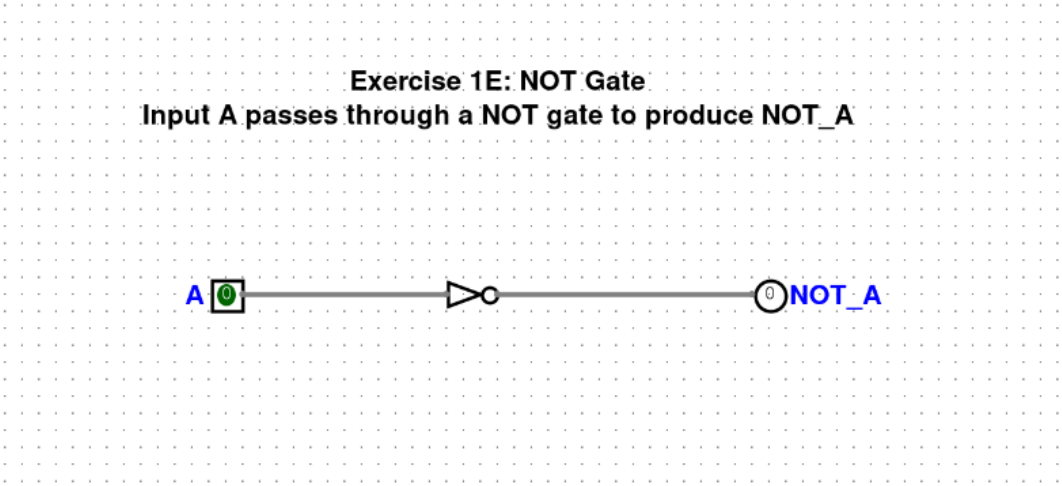

Figure 1: 1E — NOT Gate

2.1.2 8E — Single LED Control (LED Control Circuits)

This exercise demonstrates the most basic form of output visualization in a digital circuit: driving a light-emitting diode (LED) from a single control signal. The input pin serves as a push-button or toggle switch, and the LED component acts as a visual indicator of the signal state. When the input is set to 1, current flows through the wire and the LED lights up; when the input returns to 0, the LED goes dark.

There is no logic gate between the input and the LED — the connection is direct. The point of the exercise is not to perform any computation but to become familiar with the I/O components available in LogiSim. The LED component can be configured to display different colors, and it provides immediate visual feedback that is much easier to read at a glance than an output pin showing a raw 0 or 1. In more complex circuits later on, LEDs are used to monitor internal signals during debugging, so getting comfortable with them early is useful.

In a physical circuit, an LED would require a current-limiting resistor to prevent burnout, but in LogiSim the simulation abstracts away electrical characteristics and treats everything as ideal logic levels.

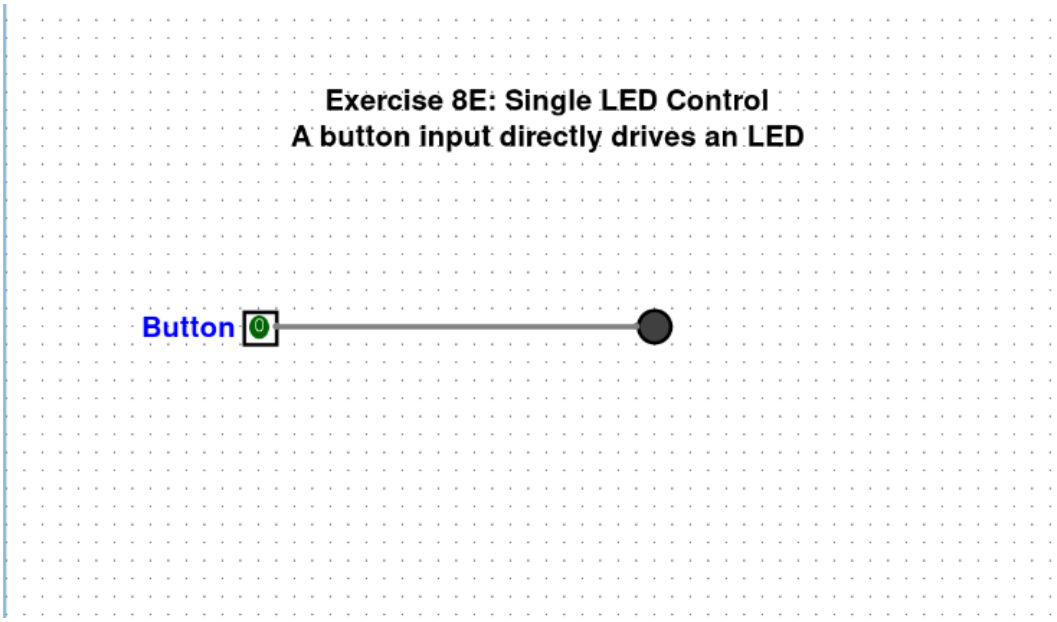

Figure 2: 8E — Single LED Control

2.2 Medium exercises

2.2.1 1M — SR Flip-Flop (Flip-Flops & Registers)

The SR (Set-Reset) flip-flop is the simplest form of memory element in digital electronics. It can store one bit of information and retain it indefinitely until explicitly told to change. The version built here uses two NOR gates with their outputs fed back into each other's inputs, forming a cross-coupled pair.

The circuit has two external inputs: S (Set) and R (Reset). When S is driven high while R stays low, the output Q latches to 1 and the complementary output $\overline{Q}$ drops to 0. When R is driven high while S stays low, Q resets to 0 and $\overline{Q}$ rises to 1. The critical behavior is what happens when both S and R are low: the flip-flop holds its previous state. This "memory" arises because the feedback loop sustains itself — each NOR gate's output reinforces the other's input. The one condition to avoid is setting both S and R high simultaneously, because that forces both NOR outputs to 0, which contradicts the assumption that Q and $\overline{Q}$ should always be complements. When both inputs then drop back to 0, the resulting state is unpredictable (it depends on propagation delays), which is why this is called the *forbidden* or *invalid* state.

In the LogiSim implementation, the top NOR gate receives S on one input and the feedback from the bottom NOR gate on the other input; its output is Q. The bottom NOR gate receives R on one input and the feedback from Q on the other; its output is $\overline{Q}$. The cross-coupling is done by routing wires diagonally from each gate's output back to the opposite gate's input.

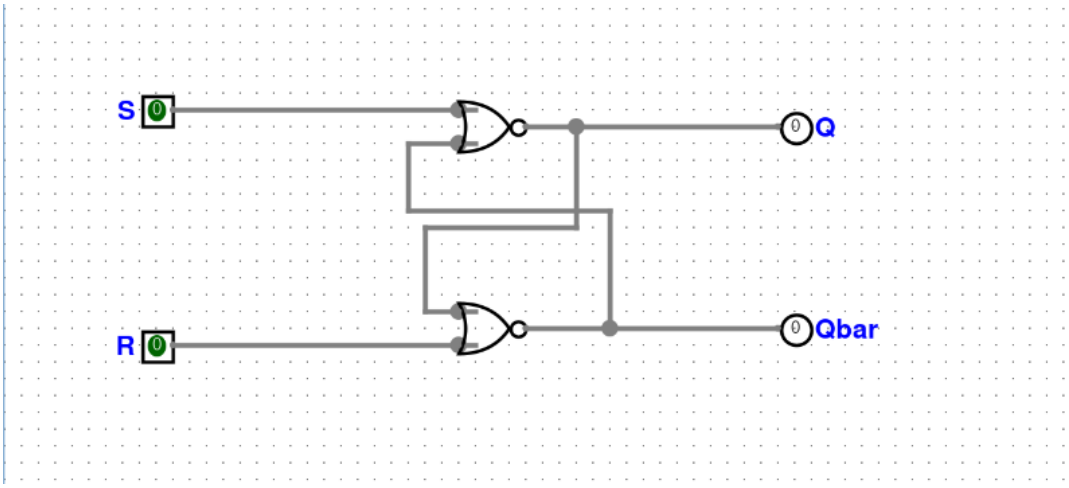

Figure 3: 1M — SR Flip-Flop

2.2.2 7M — Binary-to-Decimal Converter (Number System Conversions)

This circuit takes a 4-bit binary number as input and displays its decimal equivalent on a seven-segment indicator. The four input pins represent bits B3 (most significant) through B0 (least significant), giving a range of $0000_2 = 0_{10}$ through $1111_2 = 15_{10}$.

The conversion itself is handled by LogiSim's built-in Hex Digit Display component, which accepts a 4-bit input and illuminates the appropriate segments to show the corresponding hexadecimal digit. Since hexadecimal digits 0 through 9 are identical to their decimal counterparts, the display works as a decimal readout for any value from 0 to 9. For values 10 through 15, the display shows A through F, but the circuit still correctly converts the binary pattern into a human-readable form.

The educational value of this exercise lies in understanding positional notation. The number shown on the display equals $B3 \times 8 + B2 \times 4 + B1 \times 2 + B0 \times 1$. Toggling individual input pins and watching the display change makes this relationship very concrete. For example, turning on only B3 shows "8", turning on B3 and B0 together shows "9", and so forth.

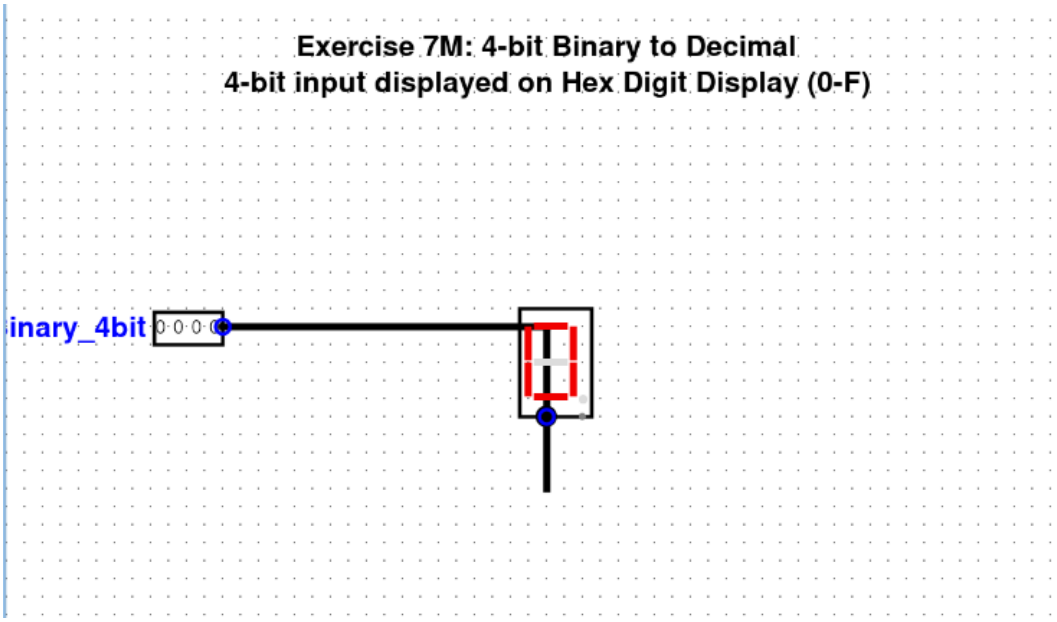

Figure 4: 7M — Binary-to-Decimal Converter

2.2.3 9M — 2-Input Multiplexer (Multiplexers & Demultiplexers)

A multiplexer (often abbreviated MUX) is a combinational circuit that selects one of several input signals and forwards it to a single output line. The selection is controlled by one or more selector inputs. In the 2-to-1 version built here, there are two data inputs (I0 and I1), one selector input (Sel), and one output (Y).

The Boolean expression implemented is $Y = I0 \cdot \overline{Sel} + I1 \cdot Sel$. When Sel is 0, the first AND gate passes I0 through (because the inverted Sel is 1) while the second AND gate blocks I1 (because Sel itself is 0). The OR gate at the end combines the two AND outputs, so Y follows I0. When Sel switches to 1, the roles reverse: the first AND gate blocks I0 and the second passes I1.

The circuit is built from four components: a NOT gate that inverts the selector, two AND gates (one combining I0 with the inverted selector, the other combining I1 with the original selector), and an OR gate that merges the two AND outputs. This structure is sometimes called the "sum of products" implementation because the output is the OR (sum) of two AND (product) terms.

Multiplexers are used everywhere in digital systems. Inside a CPU, they route data from different sources (registers, memory, ALU output) onto a shared bus depending on the current instruction. In telecommunications, they combine multiple signals onto a single channel.

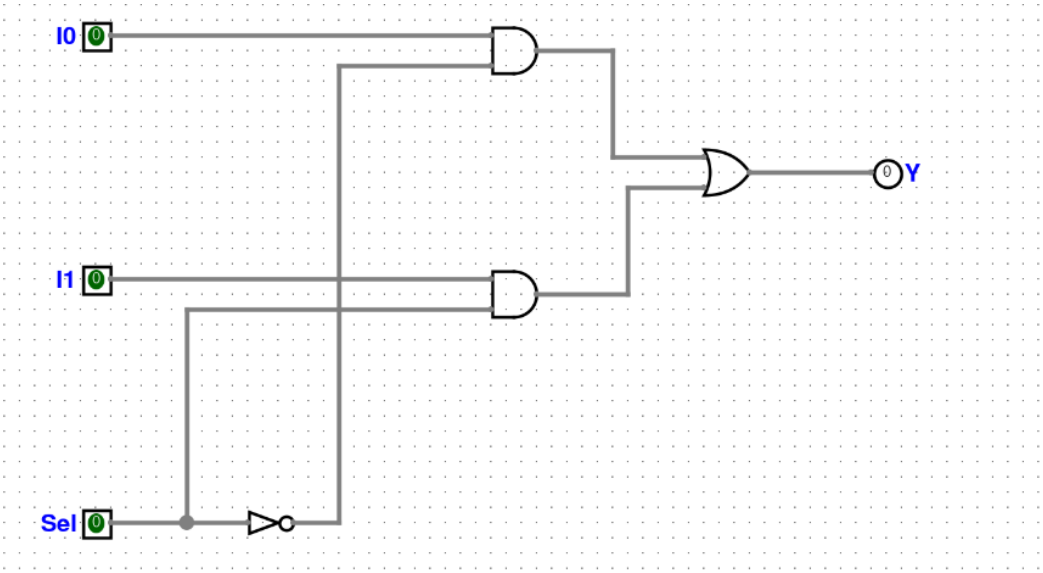

Figure 5: 9M — 2-Input Multiplexer

2.3 Hard exercises

2.3.1 1H — Clocked Register (Registers)

A register is a group of flip-flops that share a common clock signal, allowing them to capture and hold a multi-bit value simultaneously. The 4-bit clocked register built here uses four D-type flip-flops, each of which has its own data input (D0 through D3) and its own output (Q0 through Q3), but all four share the same clock line.

The D flip-flop is an edge-triggered device: it samples the value present at its D input at the exact moment the clock transitions from low to high (the rising edge), and it holds that value on its Q output until the next rising edge. Between clock edges, the output remains stable regardless of what happens on the data input. This edge-triggered behavior is what distinguishes a register from a latch, which is level-sensitive and can change output whenever the enable signal is high.

In the LogiSim circuit, the clock input pin is wired to the clock port of all four D flip-flops in parallel. The four data input pins connect to the D ports individually, and the four Q outputs connect to individual output pins. To store a value, you set the four data pins to the desired bit pattern, then toggle the clock from 0 to 1. The outputs immediately capture the data and hold it. You can then change the data pins to anything else and the outputs will not budge until the clock is toggled again.

Registers like this are the fundamental storage element in every processor. The register file inside a CPU is simply a collection of these multi-bit registers, and the entire fetch-decode-execute cycle revolves around moving data in and out of them on every clock cycle.

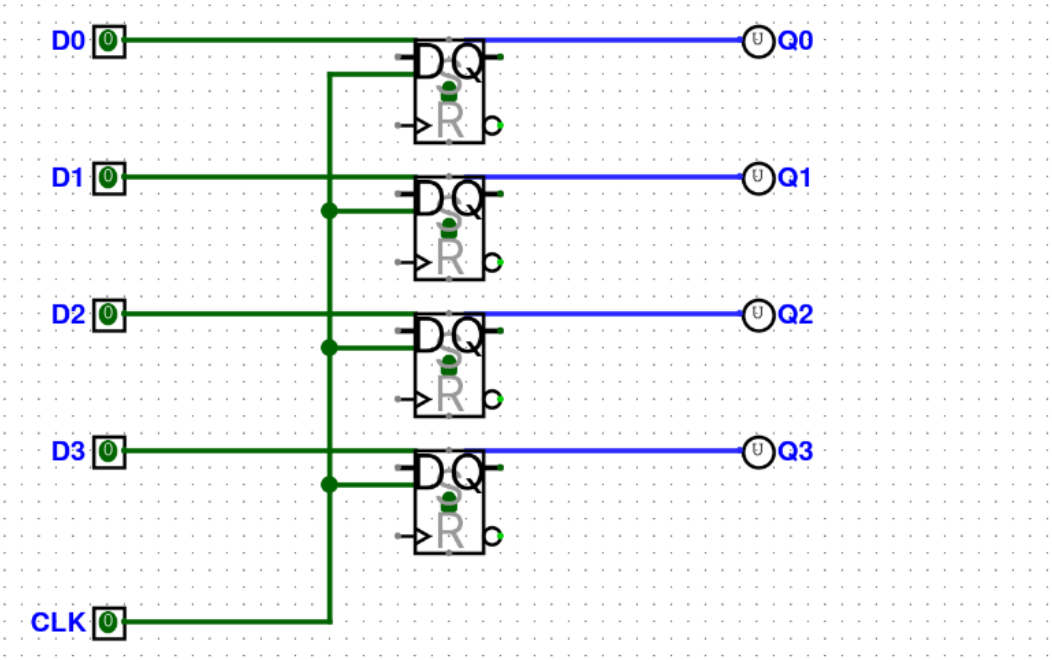

Figure 6: 1H — Clocked Register

2.3.2 2H — Bit Reversal Register (Registers)

This circuit accepts an 8-bit input and produces the same eight bits in reverse order at the output. Bit 0 of the input appears at position 7 of the output, bit 1 appears at position 6, and so on, all the way down to bit 7 appearing at position 0. The entire operation is a mirror reflection of the bit string around its center.

The implementation is purely combinational — no gates, no flip-flops, and no clock are needed. The reversal is achieved entirely through cross-wiring: the wire coming from input pin In0 is physically routed to output pin Out7, the wire from In1 goes to Out6, and so forth. Each input-output pair crosses the others in the middle of the schematic, creating a characteristic "X" pattern of crossing wires.

Despite its simplicity, bit reversal has real applications. In the Fast Fourier Transform (FFT) algorithm, the butterfly computation requires inputs in bit-reversed order, so hardware FFT accelerators include a bit-reversal stage at the front. Certain communication protocols also transmit bits in LSB-first order while the processing unit expects MSB-first, making a reversal circuit necessary at the interface.

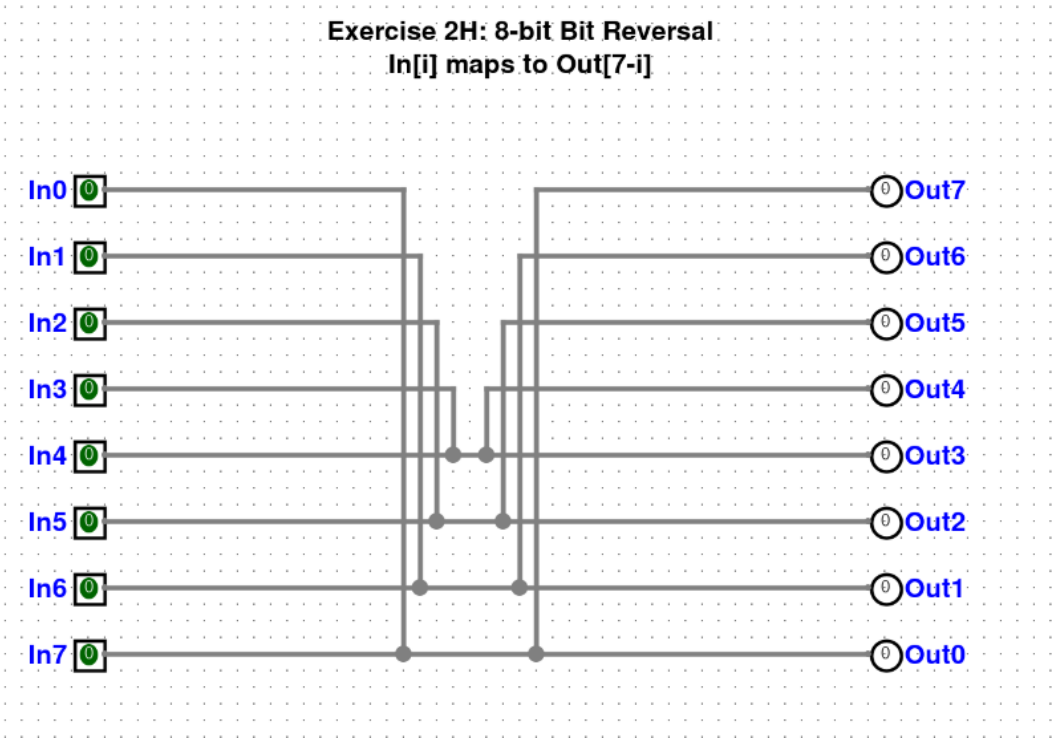

Figure 7: 2H — Bit Reversal Register

2.3.3 3H — Delay Circuit (Timing & Signal Processing)

A delay circuit introduces a controlled time lag between an input signal and the corresponding output. The version built here uses a chain of four D flip-flops connected in series, all sharing the same clock signal. The data input feeds into the first flip-flop, the output of the first feeds into the input of the second, and so on down the chain. The final output is taken from the Q pin of the fourth flip-flop.

On every rising edge of the clock, each flip-flop captures whatever its predecessor was holding. This means it takes exactly four clock cycles for a change at the input to propagate all the way to the output. If the input goes high at clock cycle t, the output will not go high until cycle $t + 4$. This creates a precise, deterministic delay that is measured in clock ticks rather than absolute time, making it independent of the clock frequency.

Such delay lines are widely used in digital signal processing. A finite impulse response (FIR) filter, for example, consists of a chain of delay elements with weighted taps at each stage. They are also used in pipeline stages inside CPUs, where data must be held back by a certain number of cycles to synchronize with other parts of the pipeline. Adding or removing flip-flops from the chain adjusts the delay length, so the circuit is easily tunable.

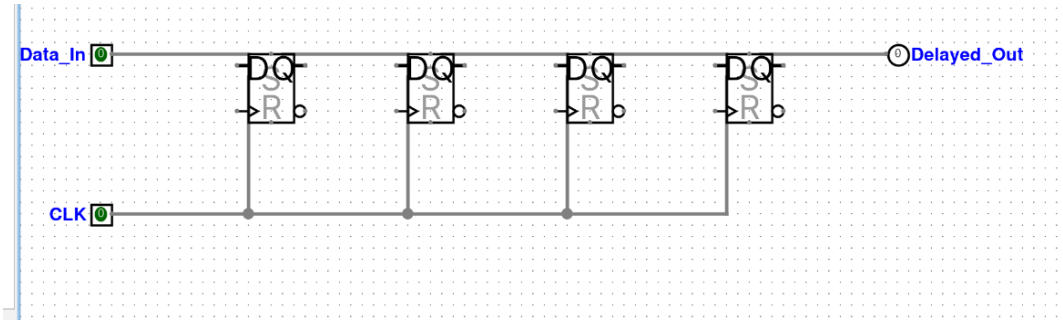

Figure 8: 3H — Delay Circuit

2.3.4 5H — 4-bit Comparator (Comparators & Encoders/Decoders)

A comparator examines two binary numbers and determines the arithmetic relationship between them. The 4-bit comparator built here accepts two 4-bit inputs, A (bits A3 down to A0) and B (bits B3 down to B0), and produces three mutually exclusive outputs: "A greater than B", "A equals B", and "A less than B". At any given moment, exactly one of these three outputs is high.

Internally, the comparison starts from the most significant bit and works downward. If A3 is 1 and B3 is 0, then A is definitely larger regardless of the lower bits, and the "greater" output is asserted. If A3 equals B3, the circuit moves on to compare A2 with B2, and so on. If all four bit pairs are equal, the "equals" output is asserted. LogiSim provides a built-in comparator component that encapsulates this cascading logic.

The inputs are wired through splitters that combine the individual 1-bit pins into a 4-bit bus, which is then fed into the comparator block. The three output pins are connected directly to the comparator's three result lines.

Comparators are essential in processors for implementing conditional branches (the "if A > B then jump" type of instruction), in sorting networks for deciding which element goes where, and in priority arbiters for determining which request to serve first.

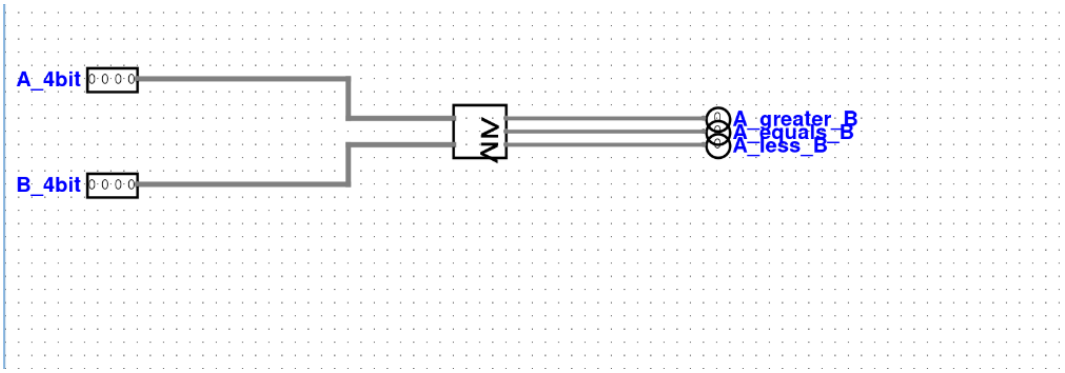

Figure 9: 5H — 4-bit Comparator

2.3.5 6H — 4-to-16 Decoder (Comparators & Encoders/Decoders)

A decoder converts a binary code into a one-hot output. The 4-to-16 decoder built here takes a 4-bit input (giving 16 possible values, from 0000 to 1111) and activates exactly one of its 16 output lines depending on the input value. If the input is 0101 (decimal 5), then output Y5 goes high and all other outputs stay low.

The underlying logic for each output line is a 4-input AND gate that checks for a specific combination of the input bits and their complements. For output Y5 (binary 0101), the AND gate would require $\overline{A3} \cdot A2 \cdot \overline{A1} \cdot A0$. Building all 16 such AND gates by hand would be tedious, so the LogiSim built-in Decoder component is used instead, but the principle is the same.

Decoders are one of the most heavily used components in digital design. In a memory system, the address decoder takes the address bits from the CPU and activates exactly one row of memory cells, allowing data to be read from or written to the correct location. Inside a CPU's control unit, an instruction decoder takes the opcode field and activates the control signals appropriate for that instruction. In display systems, a BCD-to-seven-segment decoder is a specialized version that drives the segments of a numeric display.

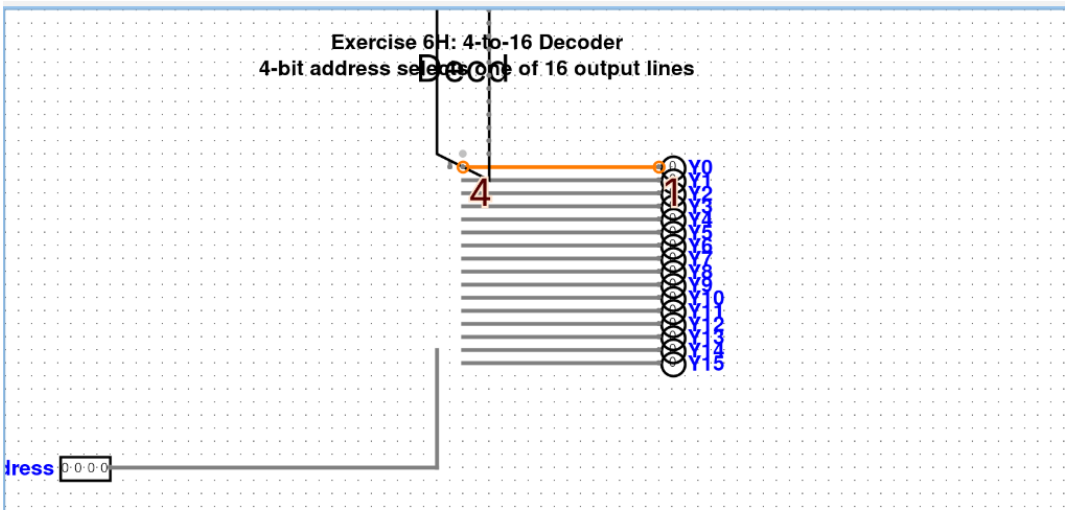

Figure 10: 6H — 4-to-16 Decoder

2.3.6 7H — 4-input Priority Encoder (Comparators & Encoders/Decoders)

A priority encoder performs roughly the inverse operation of a decoder, but with an important twist: when multiple inputs are active simultaneously, it reports only the highest-priority one. The 4-input version has four input lines (I0 through I3, with I3 being the highest priority) and produces a 2-bit binary output indicating which active input has the highest index. There is also a group signal output that indicates whether any input is active at all.

For example, if both I1 and I3 are high, the output reads 11 in binary (decimal 3), because I3 has higher priority than I1. If only I0 is high, the output reads 00 (decimal 0). If no inputs are active, the group signal goes low to indicate that the output is not valid.

Priority encoders are essential in interrupt controllers inside microprocessors. When multiple hardware devices request attention at the same time, the priority encoder determines which interrupt to service first. They also appear in floating-point units, where finding the position of the leading 1-bit in a mantissa is equivalent to a priority encoding operation. The circuit uses LogiSim's built-in Priority Encoder component, with the four input pins wired to its data ports and the output pins connected to the encoded result and the group signal.

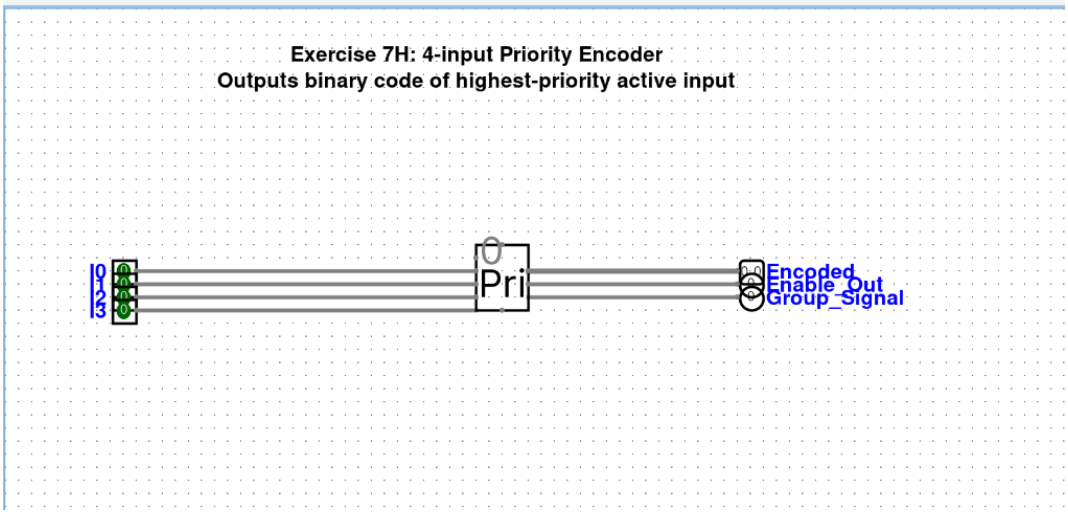

Figure 11: 7H — 4-input Priority Encoder

2.3.7 8H — 8-bit Full Adder (Arithmetic Circuits)

The full adder is the workhorse of binary arithmetic. The 8-bit version built here accepts two 8-bit operands (A and B) plus a carry-in bit and produces an 8-bit sum along with a carry-out bit. The carry-out is set to 1 whenever the result exceeds 255 (the maximum value representable in 8 bits), signaling an overflow in unsigned arithmetic.

Internally, an 8-bit adder is constructed by cascading eight single-bit full adders from the least significant bit to the most significant. Each single-bit stage computes the sum of its two input bits and the carry from the previous stage, and it produces one sum bit and one carry bit that feeds into the next stage. This is called ripple-carry addition because the carry signal "ripples" through the chain from right to left. The critical path of the circuit — the longest signal has to travel — runs through all eight carry connections, which means the circuit's propagation delay grows linearly with the number of bits. Faster designs like carry-lookahead or carry-select adders exist to mitigate this, but the ripple-carry approach is the most straightforward to understand and build.

In LogiSim, the Adder component from the Arithmetic library handles the cascading internally, so the schematic shows two 8-bit input buses, a 1-bit carry-in, an 8-bit sum output, and a 1-bit carry-out. This adder is the foundation for subtraction (by feeding the two's complement of the second operand), multiplication (which is repeated addition with shifting), and essentially all integer arithmetic in a processor.

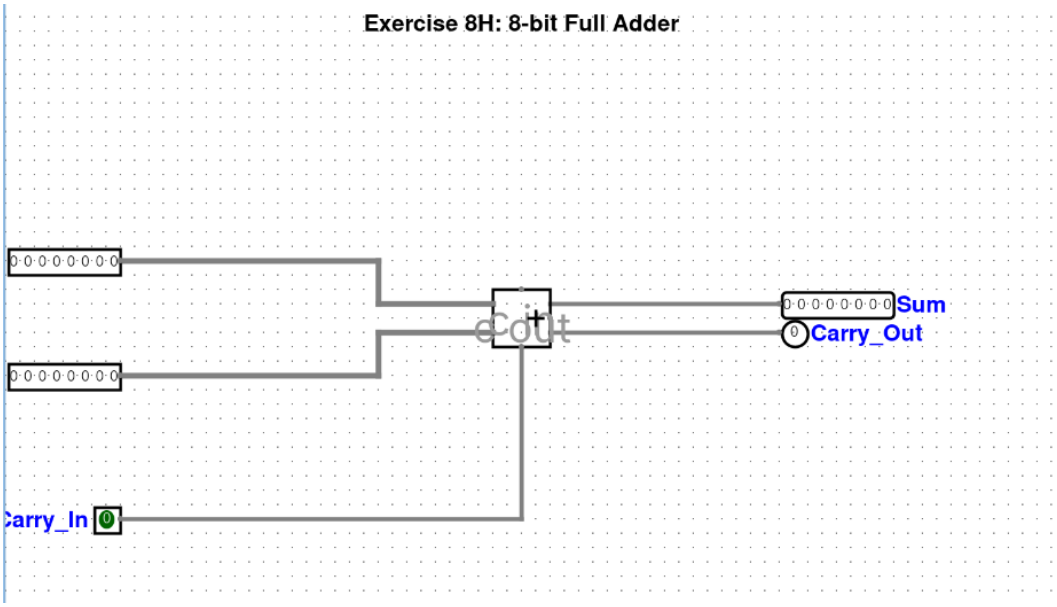

Figure 12: 8H — 8-bit Full Adder

2.3.8 15H — Left Rotation Circuit (Bitwise Operations)

A left rotation (also called a circular left shift) moves every bit in a binary number toward the most significant end by a specified number of positions. The bits that "fall off" the top wrap around to the bottom. For example, rotating the 8-bit value 10110001 left by two positions yields 11000110: the two leftmost bits (1 and 0) reappear at the two rightmost positions.

This is different from a logical left shift, where the vacated positions are simply filled with zeros and the bits shifted out are lost. Rotation preserves all the information in the original value, which makes it useful in cryptographic algorithms (many block ciphers like DES and AES use rotations extensively) and in hash functions.

The circuit uses LogiSim's Shifter component configured in left-rotate mode. The 8-bit input connects to the data port, and a 3-bit input specifies the rotation amount (0 through 7 positions, since rotating by 8 positions returns the original value). The rotated result appears on the 8-bit output. Testing the circuit is straightforward: setting the input to some recognizable pattern like 10000001 and increasing the shift amount one step at a time lets you watch the bits cycle around the byte.

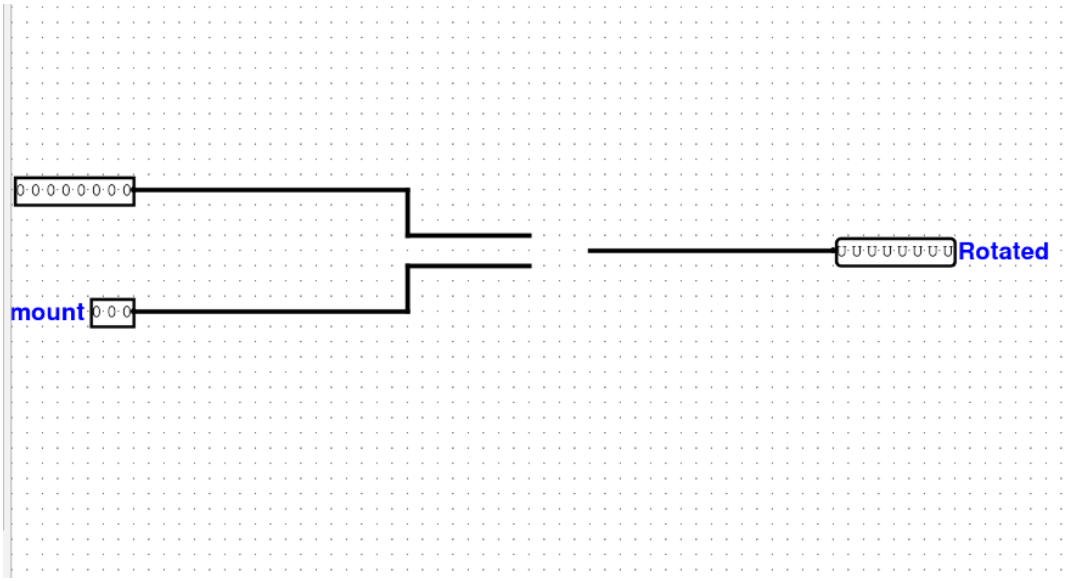

Figure 13: 15H — Left Rotation Circuit

2.3.9 16H — Bitwise Negation Circuit (Bitwise Operations)

Bitwise negation, also known as the one's complement, flips every bit in a binary number: every 0 becomes a 1 and every 1 becomes a 0. Applied to an 8-bit number, the result is the value $255 - x$ (where x is the original input). For instance, the negation of 11010010 is 00101101.

The circuit is built from eight individual NOT gates arranged in parallel, one for each bit. Each input pin connects to its own inverter, and the output of that inverter connects to the corresponding output pin. There is no interaction between the bits — each one is processed independently — which makes this a purely combinational circuit with minimal propagation delay.

Bitwise negation is a prerequisite for two's complement negation (which adds 1 after inverting), and it is used in masking operations where you need the complement of a bitmask. It also appears in error-detection schemes: comparing a data word with its stored complement is a simple way to check for single-bit corruption.

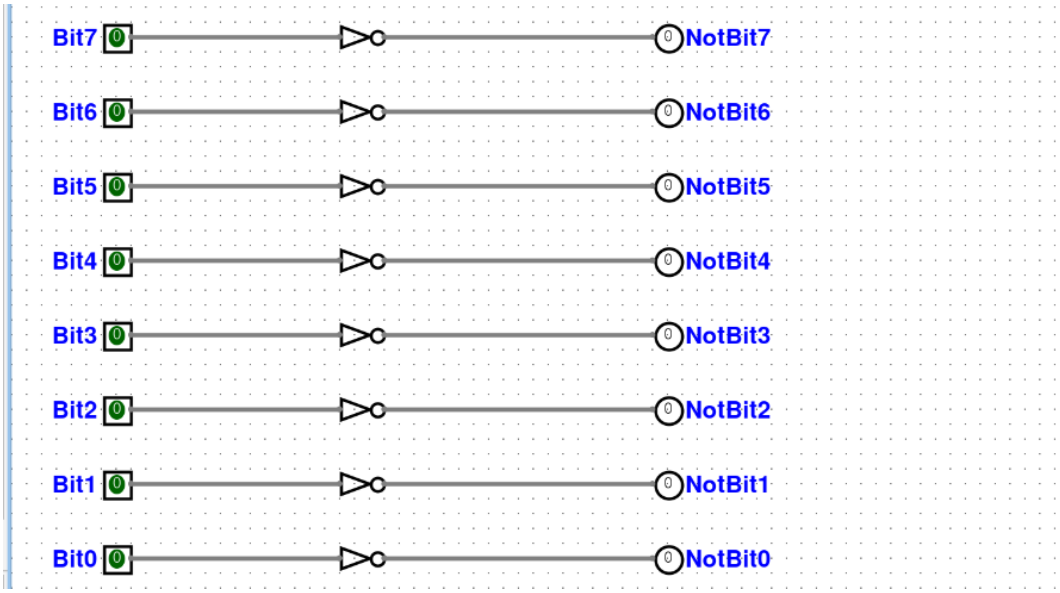

Figure 14: 16H — Bitwise Negation Circuit

2.3.10 17H — Two's Complement Circuit (Bitwise Operations)

Two's complement is the standard representation for signed integers in virtually all modern processors. To compute the two's complement of a binary number, you invert every bit (one's complement) and then add 1 to the result. The effect is to negate the number: if the original value represents $+x$, the two's complement represents $-x$ in the same fixed-width format.

For example, starting with the 8-bit value 00000101 (decimal 5), inverting gives 11111010, and adding 1 gives 11111011, which is the 8-bit two's complement representation of -5. The reason this works is that $x + \overline{x} = 11111111_2 = 255_{10}$ for any 8-bit x, so $\overline{x} + 1 = 256 - x$, and in modular arithmetic modulo 256 that is the same as $-x$.

The LogiSim circuit chains two stages. The first stage is a bank of NOT gates (identical to the bitwise negation circuit) that inverts the 8-bit input. The output of this inversion feeds into an 8-bit adder whose second operand is hardwired to the constant value 1. The adder's sum output is the final two's complement result. The carry-out of the adder is only asserted when the input is 00000000 (because $\overline{0} + 1 = 256$, which overflows), corresponding to the well-known edge case that negating zero in two's complement yields zero.

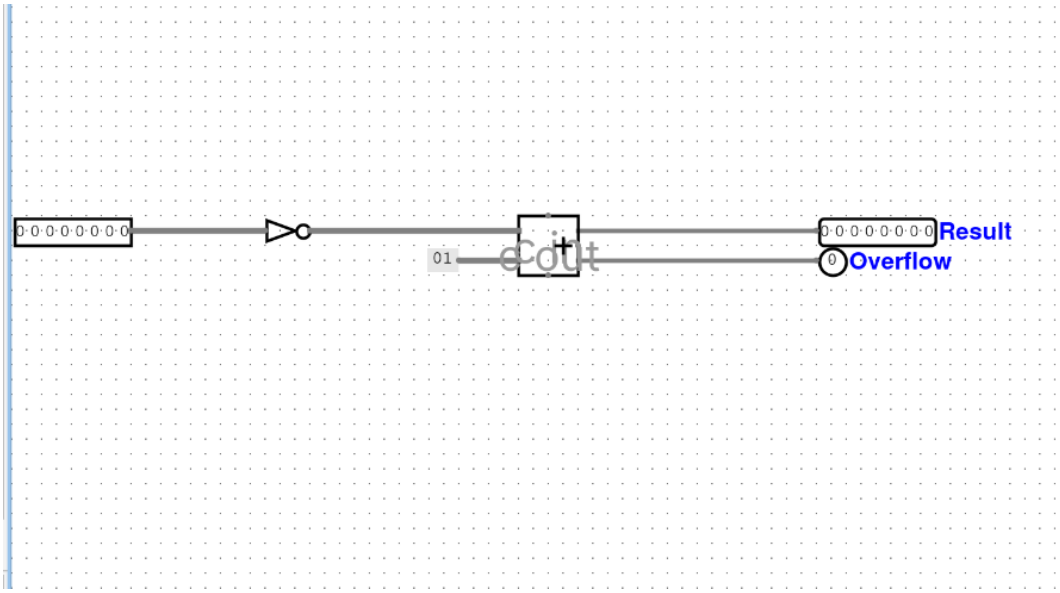

Figure 15: 17H — Two's Complement Circuit

2.3.11 18H — Binary to Decimal Conversion (Number System Conversions)

This circuit takes an 8-bit binary input (representing values from 0 to 255) and displays the equivalent decimal number on three seven-segment displays, one each for the hundreds, tens, and ones digits. The core problem being solved is the conversion from pure binary into Binary-Coded Decimal (BCD), where each decimal digit is represented by its own 4-bit group.

The conversion can be performed by the "double dabble" (shift-and-add-3) algorithm, which iteratively shifts the binary value into BCD registers and adds 3 to any BCD digit that is 5 or greater (to correct for the fact that BCD digits must not exceed 9). In the LogiSim implementation, the conversion logic feeds three separate hex digit displays, each showing one decimal digit.

Testing the circuit with known values is straightforward: setting all eight input bits to 1 (11111111) should produce "2 5 5" across the three displays, while setting the input to 01100100 (decimal 100) should show "1 0 0".

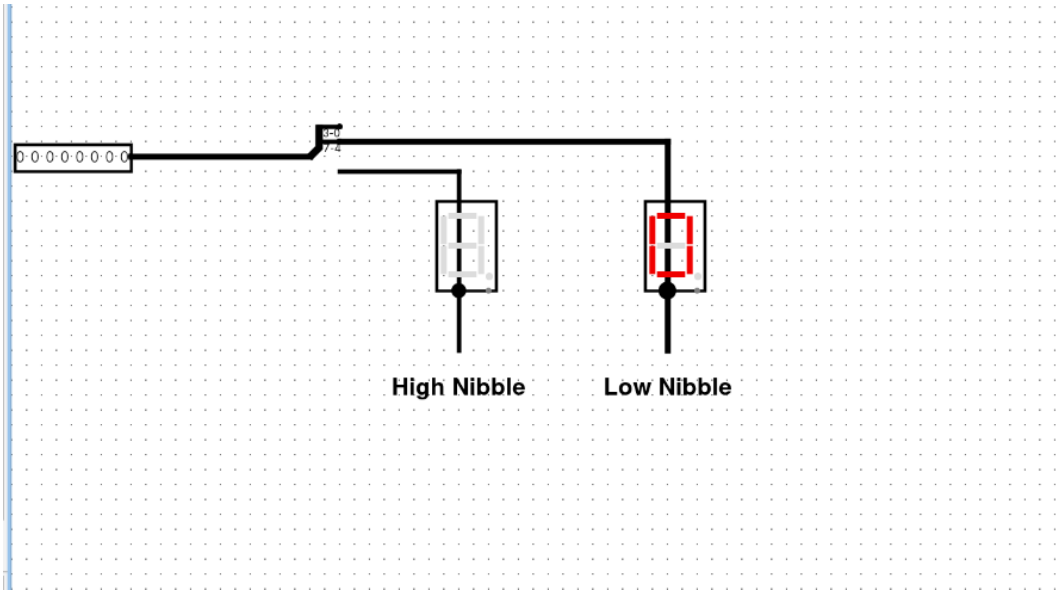

Figure 16: 18H — Binary to Decimal Conversion (8-bit)

2.3.12 22H — Binary to Hexadecimal Conversion (Number System Conversions)

Converting binary to hexadecimal is one of the simplest number-system conversions in digital electronics, because the two bases are related by a power of two: $16 = 2^4$. This means every group of exactly four binary bits maps to a single hexadecimal digit, with no arithmetic needed at all. An 8-bit number splits neatly into two 4-bit nibbles, each of which corresponds to one hex digit.

The circuit uses a Splitter component to divide the 8-bit input bus into two 4-bit buses. The lower nibble (bits 3 down to 0) connects to one hex digit display, and the upper nibble (bits 7 down to 4) connects to a second hex digit display. For example, the input 10101111 (binary) splits into 1010 (upper, which is A in hex) and 1111 (lower, which is F in hex), so the two displays read "AF".

The reason hexadecimal notation is so popular in computing is precisely this effortless mapping. Engineers can mentally convert between hex and binary without any calculation, making hex a compact shorthand for long binary strings. Memory addresses, color codes in web design (like #FF00AA), and machine-code instructions are all conventionally written in hex for this reason.

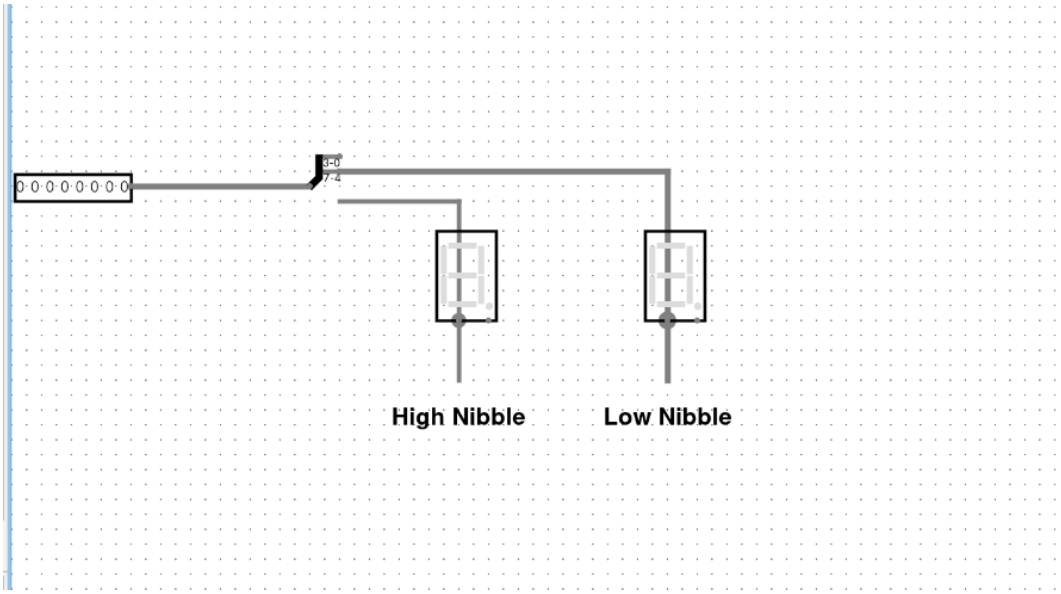

Figure 17: 22H — Binary to Hexadecimal Conversion

3 DESCRIPTION OF CIRCUIT FUNCTIONALITY

This section provides a high-level summary of the entire set of circuits, organized by functional category, and explains how the different difficulty levels build on one another.

3.1 Point allocation

The table below lists every exercise, its difficulty tier, the subcategory it belongs to, and its point contribution.

No.	Exercise	Subcategory	Level	Pts
1	1E — NOT Gate	Basic Logic Gates	Easy	0.2
2	8E — Single LED Control	LED Control Circuits	Easy	0.2
3	1M — SR Flip-Flop	Flip-Flops & Registers	Medium	0.4
4	7M — Binary-to-Decimal	Number System Conversions	Medium	0.4
5	9M — 2-Input Multiplexer	Multiplexers & Demux	Medium	0.4
6	1H — Clocked Register	Registers	Hard	0.7
7	2H — Bit Reversal Register	Registers	Hard	0.7
8	3H — Delay Circuit	Timing & Signal Processing	Hard	0.7
9	5H — 4-bit Comparator	Comparators & Enc/Dec	Hard	0.7
10	6H — 4-to-16 Decoder	Comparators & Enc/Dec	Hard	0.7
11	7H — Priority Encoder	Comparators & Enc/Dec	Hard	0.7
12	8H — 8-bit Full Adder	Arithmetic Circuits	Hard	0.7
13	15H — Left Rotation	Bitwise Operations	Hard	0.7
14	16H — Bitwise Negation	Bitwise Operations	Hard	0.7
15	17H — Two's Complement	Bitwise Operations	Hard	0.7
16	18H — Bin to Decimal (8-bit)	Number System Conversions	Hard	0.7
17	22H — Bin to Hex	Number System Conversions	Hard	0.7
			Total	**10.0**

3.2 From gates to systems

The easy-level circuits (1E and 8E) deal with the most atomic elements of digital logic. A NOT gate is a single Boolean operation, and an LED connected to a pin is the simplest possible output stage. These exercises exist to build familiarity with the LogiSim environment itself — placing components, drawing wires, running the simulation, and toggling inputs.

At the medium level the circuits begin to exhibit behavior that goes beyond simple input-to-output mappings. The SR flip-flop (1M) introduces the concept of state: the output depends not just on the current inputs but also on the history of past inputs, because the cross-coupled feedback loop creates memory. The binary-to-decimal converter (7M) introduces the idea that the same string of bits can be *interpreted* differently depending on the representation system, and that a circuit can translate between representations. The multiplexer (9M) introduces the idea of data routing, where a control signal decides which of several data paths reaches the output.

The hard-level circuits scale these ideas up considerably. Registers (1H, 2H) group multiple flip-flops to store multi-bit words. The delay line (3H) chains flip-flops in series rather than in parallel, turning a spatial arrangement into a temporal one. Comparators, decoders, and priority encoders (5H, 6H, 7H) perform multi-bit decision-making, translating between different encoding formats. The 8-bit adder (8H) and two's complement circuit (17H) bring real arithmetic into the picture, and the bitwise operations (15H, 16H) manipulate data at

the bit level. Finally, the conversion circuits (18H, 22H) bridge the gap between the binary world inside the machine and the decimal or hexadecimal world that humans prefer to read.

Together, these 17 circuits cover a representative cross-section of the functional blocks found inside any digital system. A simplified CPU, for instance, would contain registers to hold operands, an adder to perform arithmetic, a comparator to evaluate branch conditions, a decoder to interpret opcodes, a multiplexer to route data between functional units, and conversion logic to interface with external displays.

4 CONCLUSION

This laboratory provided a thorough, hands-on exploration of digital logic design through the construction and testing of 17 circuits in LogiSim, spanning three difficulty levels and covering a wide range of functional categories.

At the most basic level, the easy exercises confirmed that even the simplest circuits a single NOT gate and a direct LED connection are worth building explicitly, because they establish the workflow that every subsequent exercise depends on: place components, route wires, run the simulation, and verify the truth table against expectations. Getting this workflow right from the start made the more complex circuits much less intimidating.

The medium exercises introduced two ideas that are fundamental to everything that follows. First, the SR flip-flop demonstrated that feedback can create memory, turning a purely combinational system into a sequential one. This single concept — that a circuit can remember — is what separates trivial gate networks from the kind of logic that can actually process information over time. Second, the multiplexer showed that data can be steered by control signals, which is the basic mechanism behind every bus, every register file, and every instruction decode path in a real processor.

The hard exercises built on these foundations in several directions simultaneously. On the storage side, clocked registers showed how multiple flip-flops working together under a shared clock can capture and hold an entire word of data in one cycle, and the delay line showed how the same flip-flops arranged in series can implement precise timing control. On the decision-making side, the comparator, decoder, and priority encoder each solve a different flavor of the same problem mapping one encoding to another based on the relationships between input signals. On the arithmetic side, the 8-bit adder demonstrated carry propagation, and the two's complement circuit showed how negation reduces to inversion followed by incrementation, which is an elegant trick that lets the same adder hardware handle both addition and subtraction. On the data-manipulation side, the bitwise negation and rotation circuits showed how individual bits can be transformed in parallel, and the conversion circuits showed how a binary representation can be repackaged into decimal or hexadecimal for human consumption.

One of the most valuable takeaways from the lab is the realization that complexity in digital design is almost always managed through composition. A register is just a row of flip-flops. An adder is just a chain of full-adder cells. A two's complement unit is just an inverter followed by an adder. None of the individual building blocks are particularly difficult to understand on their own; the challenge lies in connecting them correctly and understanding the timing relationships between them.

Another important lesson is the distinction between combinational and sequential circuits. Combinational circuits produce outputs that depend only on the current inputs. Sequential circuits produce outputs that depend on both the current inputs and the internal state, which itself is a function of past inputs. This distinction maps directly to the difference between stateless and stateful computation, and understanding it is essential for designing anything more complex than a simple calculator.

Looking back at the full set of 17 exercises, a clear progression emerges. The easy tier teaches tool usage. The medium tier introduces the core abstractions of memory and selection. The hard tier applies those abstractions at scale — wider data paths, longer processing chains, and richer encoding schemes. By the end, the gap between a gate-level schematic and the functional blocks of a real processor feels much smaller than it did at the start.